

## TRABAJO PRÁCTICO INTEGRADOR 2026 de Paradigmas y Lenguajes de Programación

**Integrantes:** Mazzanti Valentina, Niveyro Lucia, Vallejos Facundo Ruben y Francisco Martin Miranda

### Sistema de Semáforos Inteligentes y Análisis Comparativo de Paradigmas

#### Objetivos del Trabajo Práctico

- **Modelar un problema del mundo real:** Traducir los requisitos funcionales de un sistema de tráfico urbano a código LISP utilizando el paradigma funcional.
- **Desarrollar autonomía y pensamiento algorítmico:** Resolver la integración de herramientas externas y el aprendizaje de una tecnología nueva con el mínimo de asistencia docente.
- **Análisis Crítico y Metacognición:** Evaluar y tipificar las funciones creadas, comprendiendo sus implicancias en la memoria y el flujo de datos, y comparar cómo diferentes lenguajes abordan un mismo problema.

Requerimiento 1:

```
1 (defun transicion2 (color-actual cambiar-a)
2   (cond
3     ((and (equal color-actual 'en-rojo) (equal cambiar-a 'verde)) (list
4       color-actual '--> 'cambiar-a-verde))
5     ((and (equal color-actual 'en-verde) (equal cambiar-a 'amarillo)) (list
6       color-actual '--> 'cambiar-a-amarillo))
7     ((and (equal color-actual 'en-amarillo) (equal cambiar-a 'rojo)) (list
8       color-actual '--> 'cambiar-a-rojo))
9     (t (list color-actual 'accion-por-defecto))
10  )
11 )
12
```

Esta función recibe dos parámetros: color-actual y cambiar-a. Luego usa cond para verificar que transición se esta realizando. Por ejemplo

Si está **en rojo** y debe pasar a **verde**: (en-rojo --> cambiar-a-verde)

Si está **en verde** y debe pasar a **amarillo**: (en-verde --> cambiar-a-amarillo)

Si está **en amarillo** y debe pasar a **rojo**: (en-amarillo --> cambiar-a-rojo)

Si no se cumple ninguna de las transiciones anteriores, devuelve una lista con el color actual y el símbolo acción-por-defecto.

Para la otra función Utiliza la función mod (módulo o resto de la división entera). Al hacer (mod unix 216), el código toma cualquier número de segundos de la hora actual y lo "encierra" en un ciclo repetitivo que va **desde el segundo 0 hasta el 215**.

Requerimiento 2:

```
(defun timer (tiempo)
  (cond
    ((and (>= (mod tiempo 216) 0) (<= (mod tiempo 216) 89)) 'ROJO)
    ((and (>= (mod tiempo 216) 90) (<= (mod tiempo 216) 95)) 'amarillo)
    ((and (>= (mod tiempo 216) 96) (<= (mod tiempo 216) 216)) 'VERDE)
  )
)
```

Acá lo que hace es usa la función (mod tiempo 216) que vimos antes para "formatear" cualquier unidad de tiempo que le pases a un rango entre **0 y 215**, y luego divide ese tiempo en tres franjas para asignar los colores.

Si el segundo actual del ciclo está entre 0 y 89 la función devuelve el símbolo 'ROJO. Esta fase dura 90 segundos.

Si el segundo del ciclo está entre 90 y 95 devuelve 'amarillo. Esta fase dura 6 segundos.

Si el segundo está entre 96 y 216, devuelve 'VERDE.

Para integrar el núcleo del paquete "Quicklisp" usamos las siguientes funciones:

- Cargamos la librería: "(ql:quickload "local-time")"
- (unix-to-timestamp unix &key (nsec 0)) -> convierte el tiempo epoch en un string con el formato 1970-01-01T00:00:00Z. Para la obtención de dicho tiempo epoch usamos la función "get-universal-time" y le restamos 2208988800, que es el resto de segundos desde el 01-01-1900 al 01-01-1970. ([Enlace en el libro de cocina](#))
- Para la obtención del formato solicitado ([2026-05-16 14:30:00]) usamos la función (local-time:format-timestring nil (local-time:now) :format '(:year "-" :month "-" :day)) ([Enlace en el libro de cocina](#))

### Requerimiento 3:

```
(defun logging (tiempo) ; forma del tiempo epoch
  (cond
    ((equal (symbol-name (timer tiempo)) "VERDE") (format nil "Tiempo <~a>: La luz a cambiado de rojo a verde" tiempo))
    ((equal (symbol-name (timer tiempo)) "AMARILLO") (format nil "Tiempo <~a>: La luz a cambiado de verde a amarillo " tiempo))
    ((equal (symbol-name (timer tiempo)) "ROJO") (format nil "Tiempo <~a>: La luz a cambiado de amarillo a rojo" tiempo))
  )
)
```

La función logging tiene como objetivo registrar e informar los cambios de estado de un semáforo según el tiempo recibido como parámetro. Para ello, recibe una variable llamada tiempo y utiliza la función timer para determinar cuál es el color actual del semáforo en ese instante.

El "symbol-name" que aparece hace que el símbolo que retorna timer se convierta en un string para apodes comparar con el equal

Dentro de la función se utiliza una estructura cond, que permite evaluar varias condiciones de manera secuencial. Primero verifica si el resultado de timer es

"VERDE". Si esto ocurre, genera un mensaje indicando que la luz cambió de rojo a verde junto con el tiempo correspondiente.

Si la condición anterior no se cumple, verifica si el color es "AMARILLO". En ese caso, devuelve un mensaje informando que la luz cambió de verde a amarillo.

Finalmente, si el color obtenido es "ROJO", devuelve un mensaje indicando que la luz cambió de amarillo a rojo.

#### Requerimiento 4a:

```
(defun duracion-ciclo (t-rojo t-amarillo t-verde); en ambos casos retorna una lista
con el total de segundos y con un simbolo si es optimo o no
2
3   (let ((total (+ t-rojo t-amarillo t-verde)))
4     (if (and (>= total 35) (<= total 150))
5         (list total 'Rango-optimo)
6         (list total 'fuera-de-rango)
7     )
8   )
9 )
10
```

La función duración-ciclo recibe tres valores numéricos.

t-rojo: tiempo del semáforo en rojo.

t-amarillo: tiempo del semáforo en amarillo.

t-verde: tiempo del semáforo en verde.

Luego calcula la duración total del ciclo del semáforo y determina si ese total está dentro de un rango considerado óptimo.

#### Requerimiento 4b:

```
(defun recomendacion-ciclo (totalCiclo); recibe como parametro la lista retornada por
la funcion "duracion-ciclo"
2   (cond
3     ((< (car totalCiclo) 35) "El ciclo es bajo, se recomienda uno que este dentro
de 35-150")
4     ((> (car totalCiclo) 150) "El ciclo es demasiado alto, se recomienda uno que
este dentro de 35-150")
5     (t "El ciclo es optimo para estandares de ingenieria de trafico")
6   )
7 )
```

La función recomendación-ciclo recibe la lista generada por duración-ciclo. Mediante car obtiene el total de segundos del ciclo y, utilizando cond, evalúa si ese valor es menor a 35, mayor a 150 o si se encuentra dentro de ese rango. Dependiendo del resultado, retorna un mensaje indicando si el ciclo es demasiado corto, demasiado largo o si cumple con los estándares óptimos de ingeniería de tráfico.

#### Requerimiento 5:

```

1 (defun ciclos-por-tiempo (minutos t-rojo t-amarillo t-verde)
2   (float (/ (* minutos 60) (car (duracion-ciclo t-rojo t-amarillo t-verde)))))
3 ]

```

La función `ciclos-por-tiempo` calcula cuántos ciclos completos de semáforo pueden realizarse en una cantidad determinada de minutos. Primero convierte los minutos a segundos, luego obtiene la duración total de un ciclo utilizando la función `duracion-ciclo`. A través de `car` extrae el total de segundos del ciclo y divide el tiempo disponible por esa duración. Finalmente, utiliza `float` para devolver el resultado como un número decimal.

#### Requerimiento 6:

```

(defun porcentaje-porColor (t-rojo t-amarillo t-verde); se pasa los tres t
  (let ((ciclo (duracion-ciclo t-rojo t-amarillo t-verde)))
    (list
      (list 'rojo (float (* (/ t-rojo (first ciclo)) 100)))
      (list 'amarillo (float (* (/ t-amarillo (first ciclo)) 100)))
      (list 'verde (float (* (/ t-verde (first ciclo)) 100)))
    )
  )
)

```

Esta función llamada `porcentaje-porColor` calcula qué porcentaje del tiempo total del ciclo le corresponde a cada color del semáforo (rojo, amarillo y verde) y te devuelve una lista con esos en formato decimal.

#### Requerimiento 7:

1-

```

* (defun transicion-de-Estado (color-actual cambiar-a)
  (cond
    ((and (eq color-actual 'en-rojo) (eq cambiar-a 'verde)) (list color-actual '--> 'cambiar-a-verde))
    ((and (eq color-actual 'en-verde) (eq cambiar-a 'amarillo)) (list color-actual '--> 'cambiar-a-amarillo))
    ((and (eq color-actual 'en-amarillo) (eq cambiar-a 'rojo)) (list color-actual '--> 'cambiar-a-rojo))
    (t (list color-actual 'accion-por-defecto))
  )
)
TRANSICION-DE-ESTADO
* (transicion-de-Estado 'en-rojo 'verde)

```

2-

```

* (defun tiempo (tiempo)
  (cond
    ((and (>= (segundoExacto tiempo) 0) (<= (segundoExacto tiempo) 89)) 'ROJO)
    ((and (>= (segundoExacto tiempo) 90) (<= (segundoExacto tiempo) 95)) 'AMARILLO)
    ((and (>= (segundoExacto tiempo) 96) (<= (segundoExacto tiempo) 215)) 'VERDE)
  )
)
TIEMPO
* (tiempo (get-universal-time))
VERDE

```

3-

```
* (defun formato-localtime (epoch)
  (local-time:format-timestring nil
    (local-time:unix-to-timestamp (- epoch 2208988800)) :format '("[ (:year 4) "-" (:month 2) "-" (:day 2) " " (:hour 2) ":" (:min 2) ":" (:sec 2) "]")))
)
FORMATO-LOCALTIME
* (defun analisis-ciclos (tiempo) ; forma del tiempo epoch
  (cond
    ((equal (symbol-name (tiempo-color tiempo)) "VERDE") (format nil "Tiempo <~a>: La luz a cambiado de rojo a verde" (formato-localtime tiempo)))
    ((equal (symbol-name (tiempo-color tiempo)) "AMARILLO") (format nil "Tiempo <~a>: La luz a cambiado de verde a amarillo " (formato-localtime tiempo)))
    ((equal (symbol-name (tiempo-color tiempo)) "ROJO") (format nil "Tiempo <~a>: La luz a cambiado de amarillo a rojo" (formato-localtime tiempo)))
  )
)
ANALISIS-CICLOS
* (analisis-ciclos (get-universal-time))
"Tiempo <[2026-06-16 01:37:14]>: La luz a cambiado de amarillo a rojo"
```

4a-

```
* (defun duracion-ciclo (t-rojo t-amarillo t-verde); en ambos casos retorna una lista con el total de segundos y con un simbolo si es optimo o no
  (let ((total (+ t-rojo t-amarillo t-verde)))
    (if (and (>= total 35) (<= total 150))
      (list total 'Rango-optimo)
      (list total 'fuera-de-rango)
    )
  )
)
DURACION-CICLO
* (duracion-ciclo 90 6 120)
(216 FUERA-DE-RANGO)
* (duracion-ciclo 50 25 25)
(100 RANGO-OPTIMO)
```

4b-

```
* (defun recomendacion-ciclo (totalCiclo); recibe como parametro la lista retornada por la funcion "duracion-ciclo"
  (cond
    ((< (first totalCiclo) 35) "El ciclo es bajo, se recomienda uno que este dentro de 35-150")
    ((> (first totalCiclo) 150) "El ciclo es demaciado alto, se recomienda uno que este dentro de 35-150")
    (t "El ciclo es optimo para estandares de ingenieria de trafico")
  )
)
WARNING: redefining COMMON-LISP-USER::RECOMENDACION-CICLO in DEFUN
RECOMENDACION-CICLO
* (recomendacion-ciclo (duracion-ciclo 90 6 120))
"El ciclo es demaciado alto, se recomienda uno que este dentro de 35-150"
* (recomendacion-ciclo (duracion-ciclo 50 25 25))
"El ciclo es optimo para estandares de ingenieria de trafico"
```

5-

```
* (defun ciclos-por-tiempo (minutos t-rojo t-amarillo t-verde)
  (float (/ (* minutos 60) (first (duracion-ciclo t-rojo t-amarillo t-verde)))))
)
CICLOS-POR-TIEMPO
* (ciclos-por-tiempo 15 90 6 120)
4.1666665
```

6-

```
* (defun porcentaje-porColor (t-rojo t-amarillo t-verde); se pasa los tres timesteps y la lista retornada por la funcion duracion ciclo
  (let ((ciclo (duracion-ciclo t-rojo t-amarillo t-verde)))
    (list
      (list 'rojo (float (* (/ t-rojo (first ciclo)) 100)))
      (list 'amarillo (float (* (/ t-amarillo (first ciclo)) 100)))
      (list 'verde (float (* (/ t-verde (first ciclo)) 100))); mientras los segundos de los colores no cambien durante en tiempo el porcentaje
    )
  )
)
PORCENTAJE-PORCOLOR
* (porcentaje-porColor 90 6 120)
((ROJO 41.666668) (AMARILLO 2.7777777) (VERDE 55.555557))
```

Iteración 2: Extensión 1

```

1 (defun segundoExacto (unix)
2   (mod (- unix 2208988800) 225)
3 )
4
5 (defun timer (tiempo)
6   (cond
7     ((and (>= (segundoExacto tiempo) 0)
8           (<= (segundoExacto tiempo) 89))
9     'ROJO)
10
11    ((and (>= (segundoExacto tiempo) 90)
12          (<= (segundoExacto tiempo) 92))
13    'AMARILLO-INTERMITENTE)
14
15    ((and (>= (segundoExacto tiempo) 93)
16          (<= (segundoExacto tiempo) 98))
17    'AMARILLO)
18
19    ((and (>= (segundoExacto tiempo) 99)
20          (<= (segundoExacto tiempo) 101))
21    'ROJO-INTERMITENTE)
22
23    ((and (>= (segundoExacto tiempo) 102)
24          (<= (segundoExacto tiempo) 221))
25    'VERDE)
26
27    ((and (>= (segundoExacto tiempo) 222)
28          (<= (segundoExacto tiempo) 224))
29    'AMARILLO-INTERMITENTE)
30  )
31 )

```

La función auxiliar `segundoexacto` lo que hace es toma una marca de tiempo (en formato de tiempo Unix) y calcula en qué segundo exacto se encuentra dentro de un **ciclo repetitivo de 225 segundos**.

La función recibe el tiempo del sistema y te devuelve simbólicamente qué luz del semáforo debería estar encendida (ROJO, VERDE, AMARILLO, etc.) en ese preciso instante dentro de su bucle infinito de 3 minutos y 45 segundos (225 segundos en total).

Extensión 2:

```

1 (defun informe (datos)
2
3   (with-open-file
4     (stream
5      "informe-ejecucion-semaforo.txt"
6      :direction :output
7      :if-exists :supersede)
8
9     (format stream
10      "Informe de Ejecución del Sistema Semafórico~%")
11
12     (format stream
13      "=====~%~%")
14
15     (mapcar
16      (lambda (dato)
17        (format stream "~a~%" dato))
18      datos)
19
20     (format stream
21      "~--- Fin del Informe ---")
22  )

```

Esta función llamada `informe` se encarga de **guardar datos en un archivo de texto externo (.txt)** creando un reporte formateado.

A continuación errores que tuvimos con las funciones:

```
[5]> (defun logging2 (tiempo) ; forma del tiempo epoch
      (cond
        ((equalp (symbol-name (timer tiempo)) "VERDE") (list "tiempo <~a>: La luz a cambiado de rojo a verde" tiempo))
        ((equalp (symbol-name (timer tiempo)) "AMARILLO") (list "Tiempo <~a>: La luz a cambiado de verde a amarillo " tiempo))
        ((equalp (symbol-name (timer tiempo)) "ROJO") (list "Tiempo <~a>: La luz a cambiado de amarillo a rojo" tiempo))
      )
    )
*** - SYSTEM::%EXPAND-FORM: (EQUALP (SYMBOL-NAME (TIMER TIEMPO)) "AMARILLO") debe ser una expresi lambda
Es posible continuar en los siguientes puntos:
ABORT      :R1      Abort main loop
```

Lo que paso aca es que faltaba un paréntesis en la primera línea de cond.

```
[1]> (defun transicion (color-actual cambiar-a)
      (cond
        ((and (equal color-actual 'en-rojo) (equal cambiar-a 'amarillo)) (list color-actual 'cambiar-a-amarillo))
        ((and (equal color-actual 'en-amarillo) (equal cambiar-a 'verde) (list color-actual 'cambiar-a-verde)))
        (t (color-actual 'accion-por-defecto))
      )
    )
TRANSICION
[2]> (transicion 'en-amarillo 'rojo)
*** - EVAL: la funci COLOR-ACTUAL no est definida
Es posible continuar en los siguientes puntos:
USE-VALUE   :R1      Input a value to be used instead of (FDEFINITION 'COLOR-ACTUAL).
RETRY       :R2      Reintentar
STORE-VALUE :R3      Input a new value for (FDEFINITION 'COLOR-ACTUAL).
ABORT       :R4      Abort main loop
Break 1 [3]> |
```

El error principal acá es un problema de sintaxis al intentar devolver una lista por defecto, lo que causó que Lisp se confundiera e intentara ejecutar una variable como si fuera una función.

```
[3]> (load "C:\\Users\\Usuario\\Desktop\\Proyecto Integrador\\Codigo_Semaforo\\semaforo.lsp")
*** - Error de Win32 267 (ERROR_DIRECTORY): The directory name is invalid.
Es posible continuar en los siguientes puntos:
ABORT      :R1      Abort main loop
Break 1 [4]> |
```

El error aca es que hubo un problema con el GNU clisp, la solución fue usar otro interprete sbcl.png.

```
* (defun timer (tiempo)
  (cond
    ((and (>= (segundoExacto tiempo) 0) (<= (segundoExacto tiempo) 89)) 'ROJO)
    ((and (>= (segundoExacto tiempo) 90) (<= (segundoExacto tiempo) 95)) 'AMARILLO)
    ((and (>= (segundoExacto tiempo) 96) (<= (segundoExacto tiempo) 215)) 'VERDE)
  )
)
debugger invoked on a SYMBOL-PACKAGE-LOCKED-ERROR in thread
#<THREAD tid=17124 "main thread" RUNNING {1104018103}>:
  Lock on package SB-EXT violated when proclaiming TIMER as a function while in
  package COMMON-LISP-USER.
See also:
  The SBCL Manual, Node "Package Locks"
Type HELP for debugger help, or (SB-EXT:EXIT) to exit from SBCL.
restarts (invokable by number or by possibly-abbreviated name):
  0: [CONTINUE] Ignore the package lock.
  1: [IGNORE-ALL] Ignore all package locks in the context of this operation.
  2: [UNLOCK-PACKAGE] Unlock the package.
  3: [ABORT] Exit debugger, returning to top level.
(PACKAGE-LOCK-VIOLATION #<PACKAGE "SB-EXT"> :SYMBOL TIMER :FORMAT-CONTROL "proclaiming ~S as a function" :FORMAT-ARGUMENTS (TIMER))
0]
```

Problema con el nombre de la función, en SBCL ya existe un símbolo llamado TIMER en un paquete por lo cual esta protegido para evitar la sobreescritura de la función.

Tenemos dos opciones. Elegir la opción "ignore-all" la cual salta la protección o la mas recomendable, cambiar el nombre de la función.

### Fase 3: Estudio Comparativo

#### Introducción a Elixir

**Elixir** es un lenguaje de programación funcional, concurrente y de código abierto creado por José Valim en 2011. Se ejecuta sobre la máquina virtual de Erlang (BEAM), lo que le permite desarrollar aplicaciones escalables, rápidas y tolerantes a fallos.

El lenguaje fue diseñado para facilitar el desarrollo de sistemas que requieren alta disponibilidad y que deben manejar una gran cantidad de usuarios o procesos simultáneamente.

#### Áreas e industrias donde se utiliza

Elixir es utilizado principalmente en:

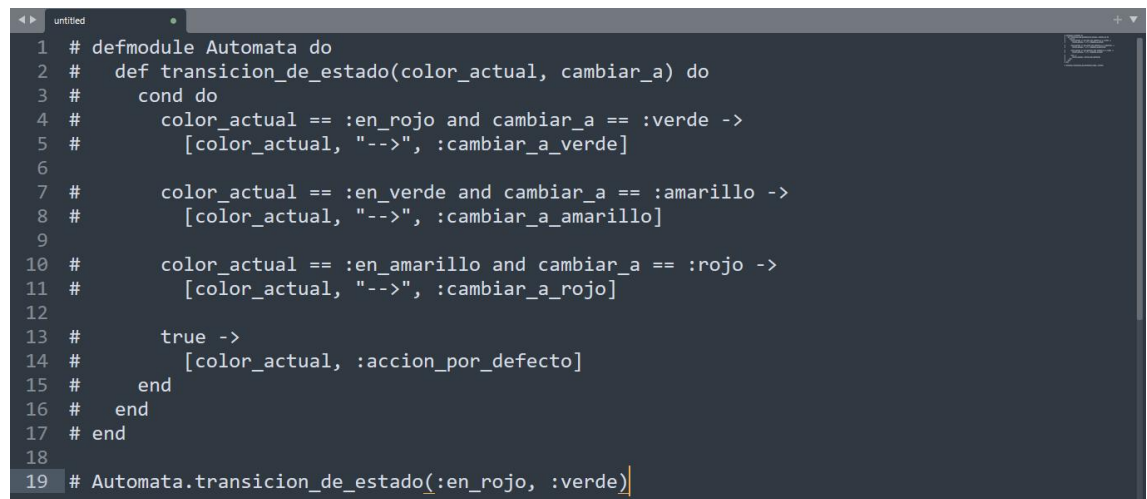
- Desarrollo de aplicaciones web.
- Sistemas de mensajería y chat en tiempo real.
- Plataformas de comercio electrónico.
- Telecomunicaciones.
- Servicios financieros y bancarios.
- Sistemas distribuidos y computación en la nube.
- Aplicaciones con alta concurrencia y disponibilidad.

#### Empresas que utilizan Elixir

Algunas empresas reconocidas que utilizan Elixir en parte de sus sistemas son:

- Discord: utiliza Elixir para manejar millones de usuarios conectados simultáneamente.
- Pinterest: emplea Elixir para mejorar el rendimiento de algunos de sus servicios.
- PepsiCo: ha utilizado Elixir en proyectos relacionados con sus plataformas digitales.
- Bleacher Report: usa Elixir para procesar grandes cantidades de información en tiempo real.
- Moz: ha incorporado Elixir en algunos de sus sistemas de análisis.

Primera función:



```
1 # defmodule Automata do
2 #   def transicion_de_estado(color_actual, cambiar_a) do
3 #     cond do
4 #       color_actual == :en_rojo and cambiar_a == :verde ->
5 #         [color_actual, "-->", :cambiar_a_verde]
6 #
7 #       color_actual == :en_verde and cambiar_a == :amarillo ->
8 #         [color_actual, "-->", :cambiar_a_amarillo]
9 #
10 #      color_actual == :en_amarillo and cambiar_a == :rojo ->
11 #        [color_actual, "-->", :cambiar_a_rojo]
12 #
13 #      true ->
14 #        [color_actual, :accion_por_defecto]
15 #    end
16 #   end
17 # end
18
19 # Automata.transicion_de_estado(:en_rojo, :verde)
```

El código define un módulo llamado Automata que simula el comportamiento de las luces de un semáforo mediante una máquina de estados. Su objetivo es validar si el cambio de un color a otro es correcto.

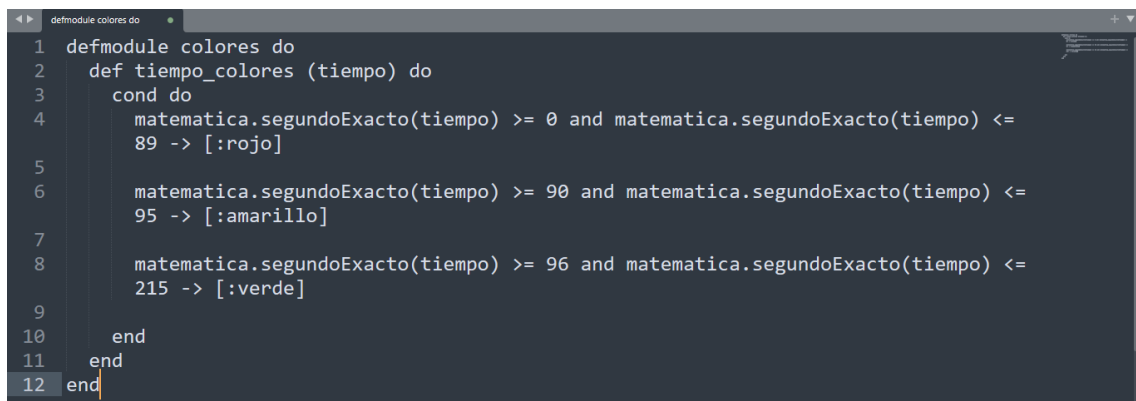
❑ La Función: `transicion_de_estado/2` recibe dos datos: el color presente (`color_actual`) y el color al que se desea pasar (`cambiar_a`).

❑ La Estructura `cond`: Evalúa las reglas en orden de arriba hacia abajo buscando la primera que sea verdadera:

1. Rojo a Verde: Si está en `:en_rojo` y se pide `:verde`, se aprueba el cambio.
2. Verde a Amarillo: Si está en `:en_verde` y se pide `:amarillo`, se aprueba el cambio.
3. **Amarillo a Rojo**: Si está en `:en_amarillo` y se pide `:rojo`, se aprueba el cambio.

❑ **El Estado por Defecto (true ->)**: Funciona como un "si no se cumple nada de lo anterior". Si intentas un salto inválido (por ejemplo, de rojo directo a amarillo), el programa lo frena y devuelve `:accion_por_defecto`.

Segunda función:



```
defmodule colores do
  def tiempo_colores(tiempo) do
    cond do
      matematica.segundoExacto(tiempo) >= 0 and matematica.segundoExacto(tiempo) <=
        89 -> [:rojo]
      matematica.segundoExacto(tiempo) >= 90 and matematica.segundoExacto(tiempo) <=
        95 -> [:amarillo]
      matematica.segundoExacto(tiempo) >= 96 and matematica.segundoExacto(tiempo) <=
        215 -> [:verde]
    end
  end
end
```

Esta función se encarga de controlar cuánto tiempo dura cada color en el semáforo basándose en los segundos.

El módulo `colores` tiene una función llamada `tiempo_colores(tiempo)`. Esta función recibe un valor de tiempo, calcula el segundo exacto usando una función externa (`matematica.segundoExacto`), y determina qué color le corresponde al semáforo según tres rangos de tiempo.

❑ Rojo (90 segundos): Si el segundo exacto está entre el 0 y el 89, el semáforo devuelve `[:rojo]`.

❑ Amarillo (6 segundos): Si el segundo exacto está entre el 90 y el 95, el semáforo devuelve `[:amarillo]`.

❑ Verde (120 segundos): Si el segundo exacto está entre el 96 y el 215, el semáforo devuelve `[:verde]`.

1. En Common Lisp, para la función `transicion` se suele usar un `cond` o `case`. En Elixir se utiliza *Pattern Matching* directo en los argumentos de la función (múltiples cláusulas). Muestren su código y expliquen por qué este enfoque es más declarativo.

```
defmodule estados do
  def transicion(:en_rojo, :verde), do: [:en_rojo, " ", :verde]
  def transicion(:en_verde, :amarillo), do: [:en_verde, " ", :amarillo]
  def transicion(:en_amarillo, :rojo), do: [:en_amarillo, " ", :rojo]
end
```

Al ser common Lisp es un lenguaje en el cual hay que decir “como” debe de realizar la tarea que se le solicita, es decir que el programador debe de dar el paso a paso para la realización de la misma. En contrario a lo que es elixir ya que es decir “que” debo de hacer, lo que hace que el código sea mas legible, fácil de testear.

- 2. Elixir utiliza el Modelo de Actores. Expliquen conceptualmente cómo se diseñaría un sistema donde cada semáforo de una esquina real sea un proceso independiente que se comunica enviando mensajes a los otros.

Diseñar este sistema en Elixir implica modelar cada esquina como un proceso independiente y aislado que posee su propio estado (color actual y temporizador). La interacción entre esquinas adyacentes elimina las variables globales y se resuelve mediante el paso de mensajes asíncronos. Cuando un semáforo agota su tiempo de luz verde, no altera el estado del semáforo vecino directamente; en su lugar, le envía un mensaje informando que iniciará su transición a rojo. El semáforo contiguo permanece a la escucha en su "buzón" y, solo al recibir la confirmación de que el cruce está liberado, altera su propio estado a verde. Este enfoque descentralizado, sumado a la arquitectura de Supervisores de Elixir, garantiza un sistema de alta disponibilidad, tolerante a fallos y fácilmente escalable a nivel urbano.

Nuestra conclusión grupal sobre usar este lenguaje elixir fue que diferencia de Lisp, donde la acumulación de paréntesis (( ( ( ) ) )) puede dificultar la lectura y el seguimiento del flujo, Elixir ofrece una sintaxis mucho más limpia, moderna y cercana al lenguaje humano. Elixir nos ofreció una alternativa moderna y altamente legible. La implementación de la máquina de estados (módulo Automata) y el control de tiempos (módulo colores) demostraron la simpleza de la estructura cond do, la cual permite evaluar reglas de negocio complejas de forma limpia y secuencial. Asimismo, el uso de átomos propios de Elixir nos permitió manejar los estados del semáforo (:en\_rojo, :en\_verde) de una manera sumamente intuitiva.

Como grupo, concluimos que Elixir no solo simplificó la traducción de la lógica algorítmica que ya teníamos en Lisp, sino que además promueve buenas prácticas de diseño de software por su claridad visual, perfilándose como una herramienta superior para el mantenimiento a largo plazo de este tipo de sistemas.



